



DEPARTMENT OF COMPUTER SCIENCE

COURSE: PROGRAMMING FUNDAMENTALS LAB

COURSE CODE: CSC-102L

SUBMITTED TO: Ms. SHANFA IRUM

SUBMITTED BY:

M. Moosa Khan (2025-SE-446)

Hassan Javaid (2025-SE-430)

Noreen Fatima (2025-SE-453)

M. Sufyan Safdar (2025-SE-451)

UET Airline Management System - Project Report

1. Project Overview

Project Title: UET Airline Management System
Domain: Console-Based Flight and Passenger Management System
Programming Language: C++
Data Storage: Text Files (Flights.txt, data.txt, Bookings.txt)
Key Features:

- Role-based access (Admin & Passenger)
- Flight management (add, remove, view)
- Passenger booking and cancellation
- Seat management with visual layout
- Booking history and reporting
- Persistent data storage

2. System Architecture

Data Structures

```
struct Flight
{
    int id;
    string origin;
    string destination;
    string date;
    string departTime;
    string arrivalTime;
    int total;
    int booked;
    int economy;
    int business;
    char** seats = nullptr;
    int rows;
    int cols;
};

struct Booking {
    int id;
    string origin;
    string destination;
    string date;
    string departTime;
    string arrivalTime;
    int fare;
    string cls;
    int seatRow;
    int seatCol;
};

struct Passenger
{
    int id;
    string firstName;
    string lastName;
    char gender;
    string email;
    string phone;
    string password;
    Booking* bookings = nullptr;
    int bookingCount = 0;
};
```

File Management

<u>File Name</u>	<u>Purpose</u>
Flights.txt	Stores flight data including seat matrix
data.txt	Stores passenger credentials and personal info
Bookings.txt	Stores all passenger bookings linked by email

3. Functional Modules

Admin Functions

- **Authentication:** Username: admin, Password: 1234
- **Functions:**

```

void admin_login();
void admin();
void changePassword();
void viewRegisteredFlights();
void addFlight();
void removeFlight();
void updatePassengerDetails();
void addPassenger();
void removePassenger();
void checkReportsPassengers();
void checkReportsFlights();
void viewCurrentPassengers();

```

Passenger Functions

- **Authentication:** ID and password (stored in data.txt)
- **Functions:**

```

void user_login();
void userMenu();
void available_flights();
void flight_booking();
void displaySeats(Flight& f);
void cancel_booking();
void generate_report();

```

System Functions

```

void loadFlights();
void savePassenger();
void appendPassenger();
void loadPassenger();
void saveFlights();
void saveBookings();
void loadBookings();

```

4. Logic

Dynamic Memory Management

- Arrays expand by 5 when capacity is reached (fcapacity, pcapacity)
- Proper cleanup using delete[] to prevent memory leaks

```

if (flightCount == fcapacity) {
    fcapacity += 5;
    Flight* temp = new Flight[fcapacity];
    for (int i = 0; i < flightCount; i++)
        temp[i] = flight[i];

    delete[] flight;
    flight = temp;
}

```

Seat Management

- Seat matrix stored as 2D char array (E = Empty, B = Booked)
- Visual display of seat layout with row/column numbering

Seat Layout (E=Empty, B=Booked)

	1	2	3	4	5
1	E	E	B	E	E
2	E	E	E	E	E
3	E	E	E	E	E
4	E	E	E	E	E

Booking System

- Booking linked to passenger via email
- Seat availability checked before booking
- Cancellation updates seat matrix and booking count

```
void flight_booking()
{
    int flightID, flightIndex;
    bool found = false;

    if (flightCount == 0) {
        cout << "No flights available!" << endl;
        return;
    }
    cout << "Enter flight ID to book: ";
    cin >> flightID;
    for (int i = 0; i < flightCount; i++) {
        if (flight[i].id == flightID) {
            flightIndex = i;
            found = true;
            break;
        }
    }
    if (!found) {
        cout << "Invalid flight ID!" << endl;
        return;
    }
    displaySeats(flightIndex);
    if (row < 0) {
        cout << "Enter row number (1-> " << flight(flightIndex).rows << "): ";
    }
    else if (row > flight(flightIndex).rows) {
        cout << "Enter row number (1-> " << flight(flightIndex).rows << "): ";
    }
    else if (col < 0) {
        cout << "Invalid row selection!" << endl;
        return;
    }
    if (flight(flightIndex).seats[row][col] == 'B') {
        cout << "Seat already booked!" << endl;
        return;
    }
    if (flight(flightIndex).booked == flight(flightIndex).total) {
        cout << "No seats available!" << endl;
        return;
    }
    int choice;
    cout << "Select class type:";
    cout << "1. Economy (" << flight(flightIndex).economy << " Pax)" << endl;
    cout << "2. Business (" << flight(flightIndex).business << " Pax)" << endl;
    cout << "Enter choice: ";
    cin >> choice;

    if (choice > 1 || choice < 0) {
        cout << "Invalid choice!" << endl;
        return;
    }
    flight(flightIndex).seats[row][col] = 'B';
    flight(flightIndex).booked++;
    cout << "Booking Successful!" << endl;
}
```

Data Persistence

- **Load on startup:** loadFlights(), loadPassenger(), loadBookings()
- **Save on modification:** saveFlights(), savePassenger(), saveBookings()

5. Strengths of the System

1. **User-Friendly Console Interface:** Color-coded and well-structured menus
2. **Role-Based Access Control:** Clear separation between admin and passenger
3. **Persistent Storage:** All changes saved to files
4. **Seat Visualization:** Graphical seat layout for easy selection
5. **Reporting Features:** Both admin and passenger can generate detailed reports
6. **Error Handling:** Validates inputs, checks for duplicates, and prevents overbooking

8. Limitations & Areas for Improvement

1. **No Data Encryption:** Passwords stored in plain text
2. **Limited Input Validation:** No regex validation for email/phone
3. **Single Admin Account:** No support for multiple admin users
4. **No Search/Filters:** Cannot search flights by date or route

9. Potential Enhancements

1. **GUI Implementation:** Using Qt or Windows Forms
2. **Database Integration:** Replace text files with SQLite/MySQL
3. **Email Notifications:** Send booking confirmations
4. **Flight Scheduling:** Recurring flights, holiday schedules
5. **Multi-City Booking:** Allow connecting flights

10. Sample Reports Generated

Passenger Booking Report

```
-----
Booking Report
-----
1:
Flight ID: 104
From: Islamabad To: Karachi
Date: 12-01-2026
Class: Business
Seat: Row 2 Seat 2      Fare Paid: 8000 PKR

Total Expenditure: 8000 PKR
-----
```

Admin Flight Report

```
--- Flight Inventory Report ---

Flight ID: 101
Route: Karachi -> Lahore
Times Booked: 0
Total Passengers carried: 5
Available Seats: 15
Status: Available
Income Generated: 0 PKR

-----

Flight ID: 102
Route: Lahore -> Islamabad
Times Booked: 0
Total Passengers carried: 3
Available Seats: 12
Status: Available
Income Generated: 0 PKR

-----

Flight ID: 103
Route: Karachi -> Islamabad
Times Booked: 0
Total Passengers carried: 2
Available Seats: 10
Status: Available
Income Generated: 0 PKR

-----

Flight ID: 104
Route: Islamabad -> Karachi
Times Booked: 1
Total Passengers carried: 7
Available Seats: 11
Status: Available
Income Generated: 8000 PKR

-----
```

11. Conclusion

The **UET Airline Management System** is a console-based application that successfully demonstrates core concepts of:

- C++ programming
- File handling
- Dynamic memory management
- Data structures
- User authentication
- Booking system logic

12. Learning Outcomes

Programming Concepts Demonstrated

1. Structures
2. Pointers & Dynamic Memory
3. File I/O
4. Arrays & Matrices
5. Functions & Modularity
6. Control Structures

Software Engineering Principles

1. **Separation of Concerns:** Admin vs Passenger modules
2. **Data Persistence:** File-based database
3. **User Interface Design:** Console menu system
4. **Error Handling:** Input validation and feedback
5. **Documentation:** In-code comments and structure